

Reflection

While experimenting with how many CPU hogs were needed to lag the interactive process a substantial amount, I realized just how fair the completely fair scheduler is. Even at 64 CPU hogs I couldn't tell if there was any sort of meaningful delay. At 128 hogs, the interactive program had a couple delays of ~3ms before evening out back to its normal ~0.06ms delay. Even with 1024 hogs, the initial run of the interactive program had a jitter of only 131ms, about half of the others had ~4ms, and the rest were normal.

After doing a quick test of my typing speed, I found it to be about 100ms per key press. (This is a very rough estimate, I got it by typing 260 characters in 26 seconds.) I tried my best to produce enough jitter to notice lag, but the most I could produce multiple times throughout one run was around 3ms. That just wasn't enough for me to notice. Maybe part of the reason for that is that I'm typing on a server that takes a million billion years to get my keypress back to me. And this isn't for a lack of trying. I had 2000 hogs running on 1 core, and was still unable to get more than a couple ms of lag consistently. And when I ran it on all the cores, the lag all but disappeared, with just a few runs having a couple ms of jitter.

Through my experimentation, it seems that the Linux CFS leans heavily towards fairness (hence the name) as it seemed the interactive program never had to wait long for its turn to run. It seemed very consistent that the highest spike in jitter occurred on the very first printf in the interactive program, which I think is likely due to the process not having fully "settled in" to the red-black tree. I guess what I mean by that is the operating system hasn't figured out that it needs more runtime. Once it gets that runtime, the jitter lessens or even disappears completely, even if there are thousands of hogs running. Since the interactive process is basically always doing I/O this or I/O that, it's barely run at all and therefore will receive a higher priority to run sooner.